1. Compare centralized and decentralized network architectures in detail

Centralized Network Architecture is a system in which all control, processing, and data storage are managed by a single central authority or server.

Decentralized Network Architecture is a system in which control, processing, and data storage are distributed across multiple independent nodes, as implemented in blockchain systems like Bitcoin and Ethereum.

---

1. Control and Authority

- **Centralized:** A single authority governs the entire system. All decisions, validations, and updates are controlled centrally. This makes administration simple but creates dependency on one entity.
- **Decentralized:** Authority is distributed among nodes. Decisions are made collectively using consensus algorithms, eliminating dependence on a single controlling entity.

---

2. Network Structure

- **Centralized:** Follows a **star topology**, where all clients are connected to one central server.
- **Decentralized:** Follows a **peer-to-peer (P2P) topology**, where nodes are interconnected and communicate directly without intermediaries.

---

3. Data Storage and Management

- **Centralized:** Data is stored in a single database or server, making it easy to update, manage, and retrieve. However, this creates a single point of failure.
- **Decentralized:** Data is replicated across multiple nodes in the form of a distributed ledger. Each node maintains a copy, ensuring redundancy and integrity.

---

4. Transparency and Auditability

- **Centralized:** Limited transparency; users must trust the authority as they cannot independently verify transactions.
- **Decentralized:** High transparency; all trans. are recorded on a public ledger and can be verified by any participant.

---

5. Security Mechanism

- **Centralized:** Security relies on firewalls, access control, and centralized authentication systems. If the central server is compromised, the entire system is at risk.
- **Decentralized:** Security is ensured through cryptographic techniques, hashing, and consensus algorithms. Even if some nodes are attacked, the system remains secure.

---

6. Single Point of Failure

- **Centralized:** Present; failure of the central server leads to total system failure.
- **Decentralized:** Absent; failure of one or more nodes does not affect the entire network.

---

7. Fault Tolerance and Reliability

- **Centralized:** Low fault tolerance due to dependence on one system. System downtime affects all users.
- **Decentralized:** High fault tolerance; redundancy ensures continuous operation even during node failures.

---

8. Performance and Speed

- **Centralized:** Faster processing since requests are handled by a single server without the need for consensus.
- **Decentralized:** Slower due to consensus mechanisms such as Proof of Work and Proof of Stake, which require validation from multiple nodes.

---

9. Scalability

- **Centralized:** Easier to scale vertically by upgrading server capacity (CPU, RAM, storage).
- **Decentralized:** Difficult to scale due to synchronization issues & increased communication overhead among nodes.

---

10. Trust Model

- **Centralized:** Requires users to trust the central authority to manage data honestly and securely.
- **Decentralized:** Trust-less environment; trust is replaced by mathematical algorithms and cryptographic proofs.

11. Cost and Infrastructure

- **Centralized:** Requires significant investment in infrastructure, maintenance, and security by the central authority.
- **Decentralized:** Costs are distributed among participants; no single entity bears the full burden.

12. Data Integrity and Immutability

- **Centralized:** Data can be modified or deleted by administrators, leading to potential manipulation.
- **Decentralized:** Data is immutable once recorded due to blockchain structure; changes require network consensus.

13. Privacy and Data Ownership

- **Centralized:** User data is controlled by the central authority and may be used for analytics or monetization
- **Decentralized:** Users have greater control over their data using private/public key cryptography.

14. Regulatory and Legal Control

- **Centralized:** Easier to regulate, monitor, and enforce laws since there is a single authority.
- **Decentralized:** Harder to regulate due to the absence of a central governing body and global distribution of nodes.

15. Decision-Making Process

- **Centralized:** Quick and efficient decision-making as it involves only one authority.
- **Decentralized:** Slower decision-making due to the need for agreement among multiple nodes.

16. Maintenance and Updates

- **Centralized:** Updates and maintenance are straightforward, handled by a central team.
- **Decentralized:** Updates require consensus and coordination among nodes, making the process complex.

17. Real-World Examples

- **Centralized Systems:** Banking systems, cloud platforms, and applications like Facebook.
- **Decentralized Systems:** Cryptocurrencies and blockchain platforms like Bitcoin and Ethereum.

18. Use Case Suitability

- **Centralized:** Suitable for applications requiring high speed, strict control, and regulatory compliance (e.g., banking).
- **Decentralized:** Suitable for applications requiring transparency, trust-less interaction, and security (e.g., cryptocurrencies, smart contracts).

Tabular Comparison (Exam-Oriented)

Feature	Centralized Network	Decentralized Network
Control	Single authority	Distributed control
Topology	Star	Peer-to-peer
Data Storage	Central server	Distributed ledger
Security	Vulnerable	Highly secure
Transparency	Low	High
Speed	High	Moderate/Low
Fault Tolerance	Low	High
Trust	Required	Trustless
Scalability	Easy	Complex
Data Integrity	Modifiable	Immutable

Diagram Instructions

- **Centralized Diagram:** one central node (server) with multiple arrows connecting to client nodes.
- **Decentralized Diagram:** Draw multiple nodes connected with each other (mesh-like struct) without any central node.

2. Describe the key characteristics of blockchain along with 3 use cases

Blockchain is a distributed, immutable digital ledger that records transactions in a sequence of blocks across multiple nodes, ensuring transparency, security, and trust without a central authority, as seen in Bitcoin and Ethereum.

Key Characteristics of Blockchain

1. Decentralization

- Blockchain eliminates the need for a central authority (like banks or servers).
- Decision-making is distributed among nodes, reducing dependency and risk of control abuse.
- This ensures no single entity can manipulate the system.

---

2. Distributed Ledger

- Every node in the network maintains a copy of the entire ledger.
- Updates are synchronized across all nodes using consensus.
- This redundancy ensures high availability and data consistency.

---

3. Immutability

- Once a block is added, it cannot be altered or deleted.
- Each block contains a cryptographic hash of the previous block, forming a secure chain.
- Any attempt to modify data breaks the chain, making tampering evident.

---

4. Transparency

- Transactions are visible to all participants (in public blockchains).
- Builds trust among users since records can be independently verified.
- Useful in systems where auditability is critical.

---

5. Security

- Blockchain uses hashing, encryption, and digital signatures.
- Consensus algorithms like Proof of Work and Proof of Stake prevent unauthorized transactions.
- Makes hacking extremely difficult due to distributed validation.

---

6. Consensus Mechanism

- All nodes must agree before a transaction is added.
- Prevents fraud such as double-spending.
- Ensures integrity and consistency across the network.

---

7. Anonymity and Privacy

- Users are identified by cryptographic addresses rather than real identities.
- Maintains privacy while still allowing transaction tracking.

---

8. Traceability

- Every transaction is linked chronologically.
- Enables tracking of assets from origin to destination.
- Particularly useful in logistics and auditing.

---

9. Fault Tolerance

- The system continues functioning even if some nodes fail.
- No single point of failure ensures high reliability.

---

10. Smart Contracts

- Self-executing programs stored on blockchain (popular in Ethereum).
- Automatically execute when conditions are met, reducing human intervention.

Use Cases of Blockchain

---

1. Cryptocurrency and Digital Payments

Explanation: Blockchain enables peer-to-peer financial transactions without intermediaries such as banks.

Working:

- A user initiates a transaction.
- It is broadcast to the network.
- Nodes validate using consensus.
- Transaction is added to a block and recorded permanently.

Example: Bitcoin allows global payments without banks.

Advantages:

- Faster international transfers
- Lower transaction fees
- No currency conversion delays
- Financial inclusion for unbanked populations

Real-world significance: Traditional banking systems may take 2–3 days for international transfers, while blockchain completes them in minutes.

---

2. Supply Chain Management

Explanation: Blockchain improves transparency and traceability of goods across the supply chain.

Working:

- Each stage (manufacturing → shipping → delivery) is recorded as a transaction.
- Data is stored in blocks and shared across stakeholders.

Example Scenario:

- A food product can be traced from farm to supermarket.
- If contamination occurs, the exact source can be identified quickly.

Advantages:

- Prevents fraud and counterfeit goods
- Enhances product authenticity
- Improves logistics efficiency
- Enables real-time tracking

Case Insight: Large companies use bchain to track food safety, reducing contamination response time from days to secs

---

3. Healthcare Data Management

Explanation: Blockchain provides a secure and unified system for storing patient medical records.

Working:

- Patient data is encrypted and stored on the blockchain.
- Access is granted using private keys.
- Multiple hospitals can access consistent records.

Advantages:

- Improved data security and privacy
- Eliminates duplicate records
- Faster diagnosis with complete history
- Better coordination among healthcare providers

Example Scenario: A patient visiting diff hospitals does not need to carry records; doctors can securely access history.

---

4. Voting Systems

Explanation: Blockchain enables secure, transparent, and tamper-proof digital voting.

Working:

- Each vote is recorded as a transaction.
- Once added, it cannot be changed.
- Results can be verified by anyone.

Advantages:

- Eliminates election fraud
- Ensures transparency
- Increases voter trust
- Enables remote voting

Example Scenario: National elections can be conducted online with guaranteed integrity.

---

5. Smart Contracts in Business

Explanation: Smart contracts automate agreements between parties.

Working:

- Conditions are coded into the blockchain.
- When conditions are met, execution happens automatically.

Example: Insurance claim is automatically processed when conditions (e.g., accident verification) are met.

Advantages:

- Reduces paperwork
- Eliminates intermediaries
- Faster execution
- Minimizes disputes

---

6. Banking and Financial Services

Explanation: Blockchain simplifies banking operations like loans, settlements, and clearing.

Advantages:

- Reduces processing time
- Improves transparency
- Lowers operational costs

Example: Cross-border settlements happen instantly instead of days.

---

7. Digital Identity Management

Explanation: Blockchain provides secure digital identities for individuals.

Working: Identity data is stored securely and accessed via cryptographic keys.

Advantages:

- Prevents identity theft
- Eliminates multiple identity documents
- Gives users control over personal data

---

8. Real Estate Management

Explanation: Blockchain simplifies property transactions and ownership records.

Working:

- Property details are stored on blockchain.
- Ownership transfers are recorded securely.

Advantages:

- Reduces fraud
- Eliminates intermediaries
- Faster property registration

---

9. Intellectual Property and Copyright Protection

Explanation: Blockchain helps creators protect ownership of digital content.

Advantages:

- Proof of ownership
- Prevents unauthorized use
- Ensures fair royalty distribution

---

10. Education and Certificates Verification

Explanation: Academic certificates can be stored and verified on blockchain.

Advantages:

- Prevents fake certificates
- Easy verification by employers
- Permanent record storage

Tabular Summary of Use Cases

Use Case	Key Benefit
Cryptocurrency	Fast, low-cost payments
Supply Chain	Transparency & tracking
Healthcare	Secure patient records
Voting	Tamper-proof elections
Smart Contracts	Automated agreements
Banking	Faster settlements
Identity	Secure digital identity
Real Estate	Fraud-free transactions

Diagram Instructions

- Draw a **blockchain structure** showing:
    - Block 1 → Block 2 → Block 3
    - Each block containing:
        - Data
        - Hash
        - Previous Hash
- Also draw **multiple nodes connected** to represent decentralization.

---

3. Elaborate cryptocurrency and its transactions

Cryptocurrency is a decentralized digital currency that uses cryptography for secure transactions and operates on a blockchain ledger without central authority, as seen in Bitcoin and Ethereum.

---

1. Core Structure of Cryptocurrency (Deep Explanation)

At its core, cryptocurrency is built on three tightly connected elements:

(a) Blockchain as the Ledger

- Every transaction is recorded in a **block**, and blocks are linked chronologically to form a chain.
- Each block contains:
    - Transaction data
    - Hash of the current block
    - Hash of the previous block
- This linking ensures that if one block is altered, the entire chain becomes invalid.

Why this matters: This structure removes the need for a bank. Instead of trusting an institution, users trust the **system's mathematical integrity**.

(b) Cryptographic Keys (Ownership System)

- Ownership in cryptocurrency is not based on identity but on **keys**:
    - **Public Key:** Like an account number (used to receive funds)
    - **Private Key:** Like a password (used to sign transactions)
- A transaction is valid only if it is signed with the correct private key.

Why this matters: Control of money = control of private key.

If the key is lost → funds are lost permanently. If stolen → funds can be transferred without reversal.

(c) Decentralized Network (Nodes)

- The network consists of thousands of nodes (computers).

- Each node:
    - Stores a copy of the blockchain
    - Verifies transactions
- No central server exists.

Why this matters:

- No single point of failure
- No authority can freeze or manipulate funds
- System is censorship-resistant

---

2. Cryptocurrency Transaction Lifecycle

A cryptocurrency transaction is not just "send money → receive money". It is a **multi-stage verification pipeline**.

Stage 1: Transaction Creation

- User A wants to send cryptocurrency to User B.
- Using a wallet, A enters:
    - B's public address
    - Amount to send

This creates a **transaction message**.

Stage 2: Digital Signing (Critical Concept)

- The transaction is signed using A's **private key**.
- This generates a **digital signature**.

Purpose of signature:

1. Proves the sender is legitimate
2. Prevents tampering
3. Ensures non-repudiation (sender cannot deny)

Without this step → system collapses (anyone could spend anyone's funds).

Stage 3: Broadcasting to Network

- The signed transaction is broadcast to all nodes.
- At this point, it is **unconfirmed**.

Think of it like announcing:
"I want to send money" — now the network must verify it.

Stage 4: Verification by Nodes

Each node independently checks:

1. **Balance Check:** Does A actually have enough funds?
2. **Signature Verification:** Is the digital signature valid?
3. **Double-Spending Check:** Has this same money already been used?

Only if all conditions pass → transaction is considered valid.

Stage 5: Transaction Pool (Mempool)

- Valid transactions are stored temporarily in a pool called **mempool**.
- Miners/validators select transactions from here.

Stage 6: Block Formation

- Multiple transactions are grouped into a block.
- The block also includes:
    - Timestamp
    - Previous block hash
    - Nonce (for mining in some systems)

Stage 7: Consensus Mechanism (Heart of the System)

The network must agree before adding the block.

Two major methods:

- **Proof of Work**
  - Miners solve complex mathematical puzzles
  - First to solve → adds block → gets reward
  - High energy consumption
- **Proof of Stake**
  - Validators are chosen based on stake (coins held)
  - Energy-efficient alternative

Why consensus is crucial: Prevents fraud and ensures all nodes agree on the same version of truth.

Stage 8: Block Addition to Blockchain

- Once validated, the block is added permanently.
- It is linked cryptographically to the previous block.

Now the transaction becomes part of **immutable history**.

Stage 9: Confirmation

- The transaction is considered confirmed after being included in a block.
- Additional blocks added after it = more confirmations = higher security.

---

3. Security and Integrity of Transactions

1. **Cryptographic Hashing**
   - Even a small change alters the hash completely
   - Makes tampering obvious
2. **Decentralized Verification**
   - No single entity can manipulate records
   - Thousands of nodes validate independently
3. **Immutability**: Once recorded, transactions cannot be reversed

---

But This Also Creates Trade-offs

- No reversal if wrong transaction is made
- No central authority for dispute resolution
- Responsibility fully on user

---

4. Practical Understanding with Real Example

Consider sending money using Bitcoin:

- You send BTC to a friend abroad
- No bank is involved
- Network verifies transaction
- Within minutes, it is confirmed
- Cost is lower than international bank transfer

Compare to banking:

- Bank: Central authority, reversible, slower
- Crypto: Trustless, irreversible, faster

---

5. Diagram You Should Draw (Very Important)

(A) Transaction Flow Diagram

Draw in sequence:

User A → Transaction → Broadcast → Nodes → Block → Blockchain → User B

(B) Block Structure

Draw a block containing: Transaction data, Hash, Previous hash

Connect multiple blocks like a chain.

4. Give an overview of the following: (i)Cryptography (ii)Hashing

Cryptography is the technique of securing information by converting it into an unreadable form using mathematical algorithms, so that only authorized parties can access it.

(i) CRYPTOGRAPHY

1. Purpose of Cryptography

Cryptography is used to ensure secure communication in the presence of attackers. It provides:

- **Confidentiality:** Ensures that only authorized users can read the data.
- **Integrity:** Ensures that data is not altered during transmission.
- **Authentication:** Verifies the identity of the sender/receiver.
- **Non-repudiation:** Prevents denial of sending a message.

---

2. Types of Cryptography (Core Concept)

(a) Symmetric Key Cryptography

- Uses a **single key** for both encryption and decryption.
- Example: AES

Working: Sender encrypts message using a secret key → Receiver decrypts using same key.

Advantages:

- Fast and efficient
- Suitable for large data

Limitation: Key distribution problem (sharing the key securely is difficult)

(b) Asymmetric Key Cryptography

- Uses **two keys**:
  - Public Key (shared)
  - Private Key (kept secret)
- Example: RSA

Working:

- Sender encrypts using receiver's public key
- Receiver decrypts using private key

Advantages:

- More secure
- Solves key distribution problem

Limitation: Slower than symmetric encryption

---

3. Role in Blockchain

- Cryptography ensures secure transactions in systems like Bitcoin.
- Digital signatures verify ownership.
- Public/private keys control access to funds.

---

4. Digital Signatures

- A cryptographic method used to verify authenticity.
- Created using private key and verified using public key.

Function:

- Confirms sender identity
- Ensures message integrity

---

5. Applications of Cryptography

- Secure communication (HTTPS)
- Online banking and payments
- Password protection
- Blockchain and cryptocurrencies

Hashing is the process of converting inp data of any size into a fixed-size output (hash value) using a mathematical func

---

(ii) HASHING

1. Nature of Hashing

- Takes input (message/data) → produces fixed-length output called **hash**.
- Performed using hash functions like SHA-256.
- It is a **one-way function** (cannot be reversed).

---

2. Key Properties of Hashing

(a) Deterministic: Same input always produces same hash.
(b) Fixed Output Size: Regardless of input size, output length remains constant.
(c) Avalanche Effect: Small change in input → completely different hash output.
(d) One-Way Function: Impossible to retrieve original data from hash.
(e) Collision Resistance: Difficult to find two inputs producing the same hash.

---

3. Working of Hashing

- Input data is passed through a hash function.
- Output is a unique hash value.

Example: "Hello" → hash value (fixed-length string)

---

4. Role in Blockchain

- Each block contains:
  - Its own hash
  - Previous block's hash
- This creates a secure chain structure.
- Used extensively in Bitcoin.

---

5. Applications of Hashing

- Password storage (hashed passwords)
- Data integrity verification
- Digital signatures
- Blockchain linking

---

6. Difference Between Cryptography and Hashing (Mini-Comparison for Extra Marks)

Feature	Cryptography	Hashing
Purpose	Secure communication	Data integrity
Reversible	Yes (with key)	No
Keys Used	Yes	No
Output	Variable	Fixed

Diagram Instructions (Important)

Cryptography Diagram: Plain Text → Encryption → Cipher Text → Decryption → Plain Text

Hashing Diagram: Input Data → Hash Function → Fixed-size Hash Output

---

5. Operation of bitcoin blockchain

Operation of Bitcoin Blockchain – Steps (List Only)

1. Transaction initiation
2. Transaction creation (input–output definition)
3. Digital signature using private key
4. Transaction broadcast to peer-to-peer network
5. Transaction validation by nodes

6. Transaction added to mempool (unconfirmed pool)
7. Selection of transactions by miners
8. Block creation (grouping transactions)
9. Block header formation (includes previous hash, nonce, Merkle root)
10. Mining process using Proof of Work
11. Puzzle solving and nonce discovery
12. Block validation by network nodes
13. Block propagation across network
14. Block added to blockchain
15. Transaction confirmation
16. Multiple confirmations for finality

---

6. Explain: (i)PoW, (ii)PoS, (iii)BFT

Consensus mechanisms are protocols that enable distributed nodes to agree on the validity of transactions and the state of the blockchain without a central authority.

(i) Proof of Work (PoW)

Proof of Work is a consensus mechanism in which miners solve complex computational puzzles to validate transactions and add new blocks to the blockchain, as used in Bitcoin.

Core Idea

- Work = solving a cryptographic puzzle
- First miner to solve → gets right to add block
- Ensures security through computational difficulty

Step-by-Step Workflow;

Step 1: Transaction Broadcast: Users initiate transactions and broadcast them to the network.
Step 2: Transaction Verification: Nodes verify transaction validity (balance, signature, double-spending).
Step 3: Block Formation: Miners collect valid transactions into a candidate block.
Step 4: Puzzle Creation: A mathematical puzzle is created using:

- Block data
- Previous block hash
- Nonce (random value)

Step 5: Mining: Miners repeatedly change the nonce to find a hash that satisfies a condition (e.g., leading zeros).
Step 6: Solution Discovery: First miner to find valid hash broadcasts the solution.
Step 7: Block Validation: Other nodes verify:

- Hash correctness
- Transactions validity

Step 8: Block Addition: Verified block is added to blockchain.
Step 9: Reward Distribution: Miner receives reward (cryptocurrency + fees).

Key Characteristics

- High security
- Energy-intensive
- Slower transaction speed

(ii) Proof of Stake (PoS) – Expanded with Workflow

Proof of Stake is a consensus mechanism where validators are chosen based on the amount of cryptocurrency they stake, as implemented in Ethereum (modern version).

Core Idea

- Stake = ownership of coins
- Higher stake → higher chance to validate

- No heavy computation required

Step-by-Step Workflow

Step 1: Transaction Broadcast: Users send transactions to the network.

Step 2: Validator Selection: Network selects a validator based on:

  - Amount of stake
  - Randomization
  - Time factors

Step 3: Block Creation: Selected validator creates a new block with valid transactions.

Step 4: Block Proposal: Validator proposes the block to the network.

Step 5: Validation by Other Nodes: Other validators verify correctness of transactions.

Step 6: Consensus Agreement: If majority agrees, block is accepted.

Step 7: Block Addition: Block is added to blockchain.

Step 8: Reward / Penalty

- Honest validator → reward
- Malicious validator → stake is reduced (slashing)

Key Characteristics

- Energy-efficient
- Faster than PoW
- Depends on stake distribution

(iii) Byzantine Fault Tolerance (BFT)

Byzantine Fault Tolerance is a consensus mechanism that allows a distributed system to reach agreement even when some nodes act maliciously or fail, based on the Byzantine Generals Problem.

Core Idea

- System tolerates faulty or malicious nodes
- Works if majority of nodes are honest
- Common in permissioned blockchains

Step-by-Step Workflow (General BFT Model)

Step 1: Transaction Broadcast: Client sends transaction request to network.

Step 2: Primary Node Selection: One node acts as leader (primary), others are replicas.

Step 3: Pre-Prepare Phase: Leader proposes a block to all nodes.

Step 4: Prepare Phase: Nodes verify the block and broadcast agreement messages.

Step 5: Commit Phase: Nodes confirm agreement after receiving enough approvals (usually 2/3 majority).

Step 6: Final Decision: Once consensus threshold is reached, block is accepted.

Step 7: Block Addition: Block is added to ledger across all nodes.

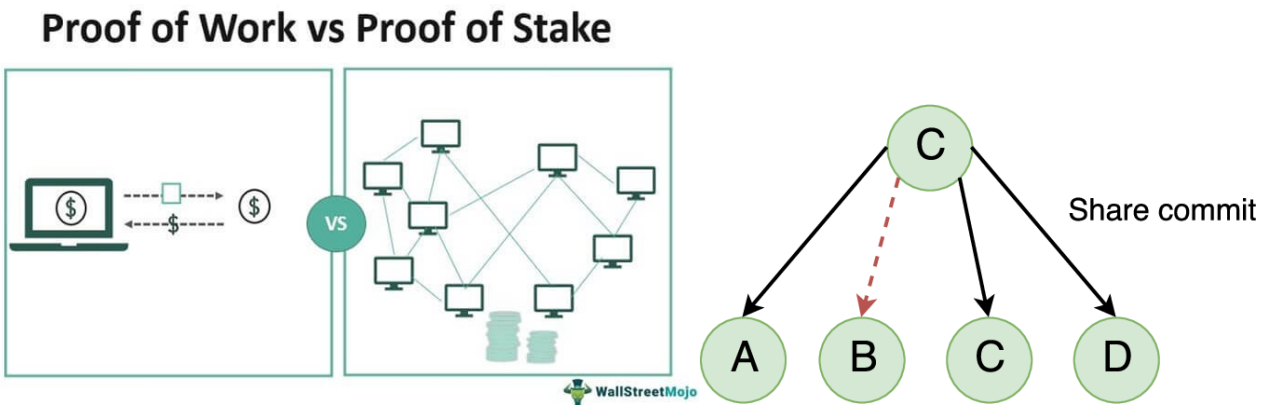
Key Characteristics

- High fault tolerance
- Fast finality (no need for multiple confirmations)
- Works well in controlled/permissioned networks

Quick Comparison (For Extra Marks)

Feature	PoW	PoS	BFT
Basis	Computation	Stake	Voting/Agreement
Energy Use	High	Low	Very Low
Speed	Slow	Moderate	Fast
Security	Very High	High	High

Feature	PoW	PoS	BFT
Use Case	Public blockchains	Modern blockchains	Permissioned systems



---

7. Describe the components of Ethereum

Ethereum is a decentralized blockchain platform that enables the execution of smart contracts and decentralized applications (DApps) using a distributed network of nodes.

Core Components of Ethereum

1. Ethereum Blockchain (Ledger Layer)

Explanation: The Ethereum blockchain is the foundational data structure that stores all transactions, smart contracts, and state changes.

Structure Details:

- Consists of a chain of blocks linked using cryptographic hashes
- Each block contains:
  - Transaction list
  - Block header (timestamp, previous hash, state root)
  - Gas usage and limits

State Concept (Very Important):

- Unlike Bitcoin (which tracks transactions), Ethereum maintains a **state-based model**
- The "state" represents:
  - Account balances
  - Smart contract data
  - Storage values

Why important: Ethereum is not just a payment system- it is a world computer storing programmable state.

---

2. Ethereum Virtual Machine (EVM)

The **EVM** is the runtime environment that executes smart contracts.

Working:

- Every node runs the EVM
- Smart contract code is executed identically on all nodes
- Ensures consistency across the network

Key Features:

- Turing-complete (can execute any logic)
- Sandbox environment (isolated execution)
- Deterministic output (same input → same output everywhere)

Why important: EVM enables Ethereum to function as a **programmable blockchain**, not just a transaction ledger.

---

3. Smart Contracts

Smart contracts are self-executing programs stored on the blockchain.

Working:

- Written in languages like Solidity

- Deployed to blockchain with an address
- Automatically execute when triggered by transactions

Example:

- If payment is received → transfer ownership automatically

Key Properties:

- Immutable after deployment
- Transparent and verifiable
- Trustless execution

Why important: They eliminate intermediaries and automate agreements.

---

4. Ether (ETH) – Native Cryptocurrency

Ether (ETH) is the native currency of Ethereum.

Functions:

- Used to pay transaction fees
- Rewards validators
- Acts as fuel for executing smart contracts

Economic Role:

- Prevents misuse of network
- Incentivizes participants

Why important: ETH powers the entire ecosystem—it is not just currency but **computational fuel**.

---

5. Gas Mechanism (Execution Cost Model)

Gas is the unit used to measure computational effort in Ethereum.

Working:

- Every operation in EVM requires gas
- Users pay gas fees in ETH

Components:

- Gas Limit → maximum computation allowed
- Gas Price → cost per unit gas

Purpose:

- Prevents infinite loops in smart contracts
- Protects network from spam

Why important: Ensures efficient resource utilization and network stability.

---

6. Ethereum Accounts (State Holders)

Ethereum has two types of accounts:

(a) Externally Owned Accounts (EOA)

- Controlled by users via private keys
- Used to send transactions

(b) Contract Accounts

- Controlled by smart contract code
- Execute automatically when triggered

Stored Data in Accounts:

- Balance (ETH)
- Nonce (transaction count)
- Storage (for contracts)

Why important: Accounts define how users and programs interact in Ethereum.

---

7. Transactions: Transactions are instructions sent by users to perform operations.

Types:

- Transfer ETH
- Deploy smart contract
- Interact with existing contract

Components:

- Sender address
- Receiver address
- Value (ETH)
- Gas limit and gas price
- Data payload

Lifecycle:

- Created → Signed → Broadcast → Validated → Included in block

Why important: Transactions are the **trigger mechanism** for all activities in Ethereum.

---

8. Nodes and Network

Nodes are computers that participate in the Ethereum network.

Types:

- Full nodes → store entire blockchain
- Light nodes → store partial data

Functions:

- Validate transactions
- Execute smart contracts
- Maintain blockchain copy

Why important: They ensure decentralization and system reliability.

---

9. Consensus Mechanism

Explanation: Ethereum uses **Proof of Stake** for validating blocks.

- Validators stake ETH
- Selected to propose and validate blocks
- Honest behavior rewarded, malicious behavior penalized

Why important: Ensures agreement across network without central authority.

---

10. Decentralized Applications (DApps)

Applications built on Ethereum using smart contracts.

Structure:

- Frontend (user interface)
- Backend (smart contracts on blockchain)

Examples:

- Finance apps (DeFi)
- Games
- NFT platforms

Why important: DApps represent real-world usage of Ethereum.

How Everything Connects (Concept Integration)

Flow:

- User (EOA) → creates transaction
- Transaction uses **gas (ETH)**
- Sent to network nodes
- Executed in **EVM**
- Smart contract logic runs
- State updated in blockchain

This integrated workflow makes Ethereum a **complete decentralized computing platform**.

---

8. Discuss DApps in detail

Decentralized Applications (DApps) are applications that run on a blockchain network instead of centralized servers, using smart contracts for backend logic and operating in a trustless, transparent, and tamper-resistant environment, commonly on platforms like Ethereum.

---

1. Core Concept of DApps (Deep Explanation)

A DApp is fundamentally different from a traditional application:

- **Traditional App:** Frontend (UI) + Backend (central server + database)
- **DApp:** Frontend (UI) + **Smart Contracts (blockchain backend)**

Instead of storing data in centralized databases, DApps store logic and state on the blockchain, making them:

- Decentralized
- Trustless (no need to trust a company)
- Transparent (code and transactions are visible)

Key Insight: DApps remove intermediaries and replace them with **code-based trust**.

---

2. Architecture of DApps (Very Important Core)

A DApp consists of three tightly integrated layers:

(a) Frontend (User Interface)

- Built using web/mobile technologies (HTML, JS, React, etc.)
- Interacts with blockchain through APIs or libraries
- Example: wallet interfaces, dashboards

Role: Acts as the interaction layer between user and blockchain.

(b) Smart Contracts (Backend Logic)

- Self-executing programs stored on blockchain
- Written in languages like Solidity (on Ethereum)
- Define rules and logic of the application

Example: If user sends payment → automatically transfer ownership of asset

Key Property: Once deployed → **immutable and tamper-proof**

(c) Blockchain (Data Layer)

- Stores:
    - Transactions
    - Smart contract code
    - Application state

Role: Acts as a distributed database accessible to all participants.

---

3. Working of a DApp (Step-by-Step Flow)

Step 1: User Interaction: User interacts with frontend (e.g., clicks “Send” or “Buy”).

Step 2: Transaction Creation: Frontend creates a transaction request.

Step 3: Wallet Authorization: User signs transaction using private key (via wallet).

Step 4: Broadcast to Network: Transaction is sent to blockchain nodes.

Step 5: Smart Contract Execution: Contract logic executes inside blockchain (via EVM).

Step 6: Consensus Validation: Network validates transaction using consensus.

Step 7: State Update: Blockchain state is updated permanently.

Step 8: Result Returned: Updated data is reflected in frontend UI.

Key Insight: Backend execution happens on blockchain → not on centralized servers.

---

4. Key Characteristics of DApps (Expanded)

1. Decentralization

- No central authority controls the application.
- Data is distributed across nodes.

2. Open Source

- Source code is often publicly available.
- Anyone can verify or contribute.

3. Immutability

- Smart contracts cannot be altered once deployed.

- Ensures reliability and trust.

4. Transparency

- All transactions are visible on blockchain.
- Enhances accountability.

5. Trustless Operation: Users rely on code, not intermediaries.

6. Tokenization

- Many DApps use tokens for:
    - Payments
    - Rewards
    - Governance

7. Security: Protected using cryptography and distributed validation.

---

5. Advantages of DApps (Deep Explanation)

1. Elimination of Intermediaries

- No need for banks, brokers, or third parties.
- Reduces cost and increases efficiency.

2. High Reliability: No single point of failure due to decentralization.

3. Data Integrity: Data cannot be manipulated or deleted.

4. User Control: Users control their own data and assets.

5. Global Accessibility: Anyone with internet can access DApps.

---

6. Limitations of DApps (Important for Balance)

1. Scalability Issues: Blockchain networks can be slow under heavy load.

2. High Transaction Cost: Gas fees can be expensive during congestion.

3. Complexity: Difficult to develop and maintain.

4. Irreversibility: Mistakes in transactions cannot be undone.

5. User Experience: Less user-friendly compared to traditional apps.

---

7. Real-World Use Cases of DApps (Expanded)

1. Decentralized Finance (DeFi)

Explanation: Financial services without banks.

Examples:

- Lending and borrowing
- Decentralized exchanges

Impact:

- Removes intermediaries
- Provides global financial access

---

2. Gaming DApps

Explanation: Blockchain-based games where users own in-game assets.

Features:

- NFTs represent items
- Players can trade assets

---

3. Supply Chain DApps

Explanation: Track goods across supply chain.

Benefits:

- Transparency
- Fraud prevention
- Real-time tracking

---

4. Social Media DApps

Explanation: Decentralized platforms where users control their content.

Benefit:

- No censorship
- No centralized data control

---

5. NFT Platforms
Explanation: DApps for creating and trading digital assets.
Use: Art, music, collectibles

---

8. DApps vs Traditional Apps (Exam Table)

Feature	Traditional App	DApp
Backend	Central server	Blockchain
Control	Single authority	Distributed
Transparency	Low	High
Trust	Required	Trustless
Data Storage	Central DB	Distributed ledger

---

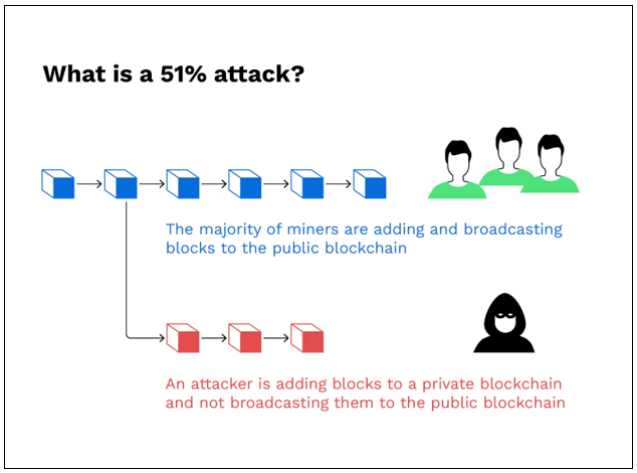
9. Explain various attacks on blockchain and its mitigation techniques

A **blockchain attack** is any attempt by an adversary to exploit weaknesses in a blockchain network (consensus, network layer, smart contracts, or user layer) to manipulate transactions, disrupt operations, or gain unfair advantage in systems like Bitcoin and Ethereum.

---

1. 51% Attack (Major Consensus Attack – Core Topic)
Explanation

- Occurs when a single entity controls **more than 50% of network hashing power or stake**. Attacker can:
  - Rewrite recent blocks
  - Reverse transactions (double spending)
  - Block other transactions



Step-wise Attack Flow

1. Attacker gains majority computational power
2. Starts mining a private chain
3. Makes a transaction on public chain (e.g., payment)
4. Simultaneously builds a longer private chain
5. Releases private chain → network accepts it as valid
6. Original transaction gets reversed → attacker keeps funds

Impact

- Loss of trust
- Double-spending fraud
- Network instability

Mitigation Techniques

- Increase network decentralization (more nodes/miners)
- Use hybrid or stronger consensus mechanisms
- Increase confirmation time before accepting transactions
- Economic disincentives (high cost to attack)

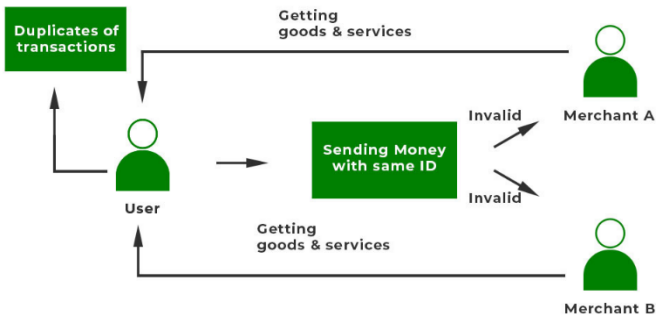
---

2. Double Spending Attack
Explanation

- Same cryptocurrency is spent more than once.
- Exploits delay in transaction confirmation.

Attack Flow

1. User sends transaction to merchant
2. Before confirmation, attacker sends conflicting transaction



3. Network eventually confirms one transaction
4. Merchant may lose funds if unconfirmed transaction was trusted

Types

- **Race attack-** A race attack is a type of double-spending attack, frequently seen in proof-of-work (PoW) networks, where a vendor accepts a transaction before it is confirmed in a block
- **Finney attack**- A Finney attack is a sophisticated, miner-specific double-spending attack on blockchain networks, named after Hal Finney. It works when a miner pre-mines a transaction—sending coins from their wallet (A) to another (B)—into a valid block but waits to broadcast it. The miner then immediately spends the same coins again before the network confirms the first transaction

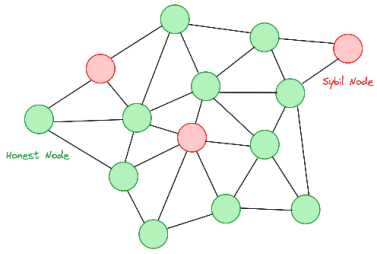
Mitigation

- Wait for multiple confirmations
- Use secure consensus (e.g., Proof of Work)
- Real-time monitoring systems

---

3. Sybil Attack

Explanation: Attacker creates multiple fake identities (nodes) to influence the network.



Impact

- Gains disproportionate control
- Disrupts consensus or routing

Mitigation

- Identity validation mechanisms
- Resource-based participation (PoW / PoS)
- Node reputation systems

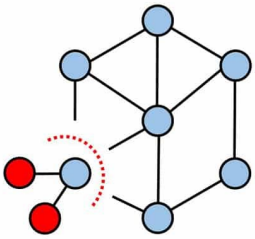
---

4. Eclipse Attack

Explanation: Attacker isolates a node by controlling all its incoming/outgoing connections.

Attack Flow

1. Attacker surrounds victim node with malicious nodes
2. Victim receives false blockchain information
3. Attacker manipulates transactions or delays confirmations



Impact

- Enables double spending
- Network partitioning

Mitigation

- Randomized peer selection
- Limiting incoming connections
- Using trusted nodes

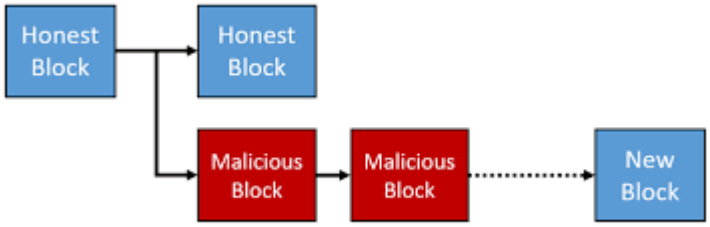
---

5. Selfish Mining Attack

Explanation: Miner withholds mined blocks instead of broadcasting immediately.

Attack Flow

1. Miner mines block but keeps it private
2. Builds longer private chain
3. Releases it strategically
4. Gains more rewards than honest miners



Impact

- Unfair reward distribution
- Centralization risk

Mitigation

- Modify mining reward rules
- Encourage honest mining
- Improve block propagation

---

6. Smart Contract Attacks

(a) Reentrancy Attack

Explanation: Attacker repeatedly calls a contract before previous execution finishes.

Example Case: Famous DAO attack on Ethereum

Mitigation

- Use checks-effects-interactions pattern
- Implement reentrancy guards
- Proper contract auditing

(b) Integer Overflow/Underflow

Explanation: Arithmetic errors allow attackers to manipulate values.

Mitigation

- Use safe math libraries
- Input validation

(c) Denial of Service (DoS) in Contracts

Explanation: Contract becomes unusable due to malicious inputs or loops.

Mitigation

- Limit gas usage
- Avoid unbounded loops

---

7. Routing Attack (Network Layer Attack)

Explanation: Attacker intercepts or delays network communication.

Impact

- Delayed transactions
- Double-spending opportunities

Mitigation

- Secure routing protocols
- Network monitoring
- Encryption of communication

---

8. Phishing and Private Key Attacks (User-Level Attack)

Explanation: Users are tricked into revealing private keys.

Impact

- Complete loss of funds
- Unauthorized transactions

Mitigation

- User awareness
- Hardware wallets
- Multi-factor authentication

---

9. Dusting Attack

Explanation: Small amounts of crypto are sent to wallets to trace identity.

Purpose: De-anonymize users

Mitigation

- Avoid reusing addresses
- Use privacy-focused wallets

Integrated Understanding (Important for High Marks)

Blockchain attacks occur at **different layers**:

- **Consensus Layer:** 51% attack, selfish mining
- **Network Layer:** Eclipse, routing attack
- **Application Layer:** Smart contract bugs

- **User Layer:** Phishing, key theft
  A strong blockchain system must secure **all layers simultaneously**.

---

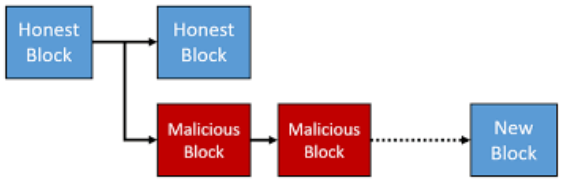
10. Elucidate: (i)Selfish Mining (ii) 51% Attack

(i) Selfish Mining

Selfish Mining is a strategy in which a miner or mining pool deliberately withholds newly mined blocks instead of broadcasting them immediately, in order to gain an unfair advantage over honest miners in networks like Bitcoin.

Core Idea

- Honest miners: publish blocks immediately
- Selfish miner: **keeps blocks private** and releases them strategically
- Goal: create a **longer private chain** and force others' work to be wasted



Step-by-Step Workflow

Step 1: Normal Mining Begins: All miners compete to solve the Proof of Work puzzle.

Step 2: Selfish Miner Finds a Block

- Instead of broadcasting, attacker **keeps the block private**.
- Private chain length = 1
- Public chain length = 0

Step 3: Honest Network Mines a Block

- Now:
  - Public chain length = 1
  - Private chain length = 1

Step 4: Strategic Release

- Attacker immediately publishes the hidden block.
- Network sees **two competing chains (fork)**.

Step 5: Network Chooses Chain

- Some nodes follow attacker's chain (due to propagation advantage).
- If attacker mines next block faster → private chain becomes longer.

Step 6: Chain Domination

- Once attacker's chain is longer:
  - Network accepts it as valid chain
  - Honest miners' blocks are discarded

Step 7: Reward Gain

- Attacker gets more rewards
- Honest miners lose effort and rewards

Impact of Selfish Mining

- Unfair reward distribution
- Encourages centralization (large pools dominate)
- Weakens trust in consensus mechanism

- Reduces overall network efficiency

Mitigation Techniques

- Improve block propagation speed
- Penalize delayed block broadcasting
- Modify reward distribution rules
- Increase decentralization of mining power

Key Insight: Even without 51% power, a miner can gain disproportionate rewards by exploiting timing and n/w delays.

(ii) 51% Attack

A **51% Attack** occurs when a single entity controls more than half of the network's computational power or stake, allowing it to manipulate the blockchain in systems like Bitcoin.

Core Idea

- Blockchain follows **longest chain rule**
- Majority power = control over which chain becomes valid
- Attacker can override honest miners

Step-by-Step Workflow

Step 1: Gain Majority Control

- Attacker acquires >50% of:
    - Hash power (PoW) OR
    - Stake (PoS)

Step 2: Start Mining Privately: Attacker begins building a **private chain** secretly.

Step 3: Perform Public Transaction

- Attacker sends cryptocurrency to a merchant.
- Merchant accepts transaction after initial confirmation.

Step 4: Continue Private Mining: Attacker continues mining secretly, excluding the transaction.

Step 5: Private Chain Becomes Longer: Since attacker has majority power, private chain grows faster.

Step 6: Chain Release: Attacker releases private chain to network.

Step 7: Chain Replacement

- Network accepts attacker's chain (longest chain rule).
- Original transaction disappears.

Step 8: Double Spending

- Attacker keeps both:
    - Goods/services from merchant
    - Original cryptocurrency

Capabilities of Attacker

- Reverse transactions
- Double spend coins
- Prevent transaction confirmations
- Block other miners

Limitations

- Cannot create new coins arbitrarily
- Cannot alter older blocks beyond a point
- Cannot access others' private keys

Impact

- Severe loss of trust
- Financial losses
- Network instability

Mitigation Techniques

- Increase network decentralization
- Encourage distributed mining pools
- Use stronger consensus mechanisms
- Wait for multiple confirmations before accepting transactions

Key Insight: Security of blockchain depends on **no single entity controlling majority power**.

Quick Comparison

Feature	**Selfish Mining**	**51% Attack**
Power Required	Less than 50%	More than 50%
Goal	Increase mining rewards	Control blockchain
Method	Withholding blocks	Rewriting chain
Impact	Unfair rewards	Double spending